

JavaScript Logic Mastery Guide

The hidden math mechanics behind loops, digit manipulation, and pattern recognition

Part 1 — The Secret Toolbox

Before writing code, understand your tools. These operators and methods are the foundation of interview-level algorithm logic.

1. The Modulo Operator (%) — "The Scissor"

`%` doesn't give you a division answer — it gives you the **remainder** left over after division.

If you modulo any base-10 number by `10`, the remainder is always the last digit.

Example: `345 % 10` → 10 goes into 345 exactly 34 times ($34 \times 10 = 340$). What's left over? `5`.

2. `Math.trunc()` vs `Math.floor()` — "The Eraser"

Dividing an integer in JS doesn't round down automatically — `345 / 10` gives you `34.5`.

- `Math.trunc(34.5)` chops off the decimal entirely → `34`. It erases the last digit.
- `Math.floor()` rounds down. Same result for positive numbers, but differs for negatives: `Math.floor(-3.5)` → `-4`, while `Math.trunc(-3.5)` → `-3`.

For logic building, `Math.trunc()` is generally safer and cleaner.

3. Modulo 2 — "The Filter"

- Even number $\div 2$ → remainder `0` (e.g. `4 % 2 === 0`)
- Odd number $\div 2$ → remainder `1` (e.g. `7 % 2 === 1`)

Part 2 — How to Dry-Run Code in Your Head

Interviews test how you think, not just how fast you type. To trace a loop mentally or on paper, use a **State Tracking Table** — never hold all changing variables in working memory at once.

The Mental Rules

1. **Initialize your variables** — write down their starting values outside the loop.
2. **Evaluate the condition** — true? enter the loop. false? skip to the end.
3. **Execute and update** — change variable values line by line.
4. **Loop back** — re-evaluate the condition with the new values.

Example Trace — Digit Sum Loop (n = 42)

Variables Before Loop: n = 42, sum = 0

[Iteration 1]

Is n > 0? (42 > 0 is true) → Enter Loop
digit = 42 % 10 → digit = 2
sum = sum + digit → sum = 0 + 2 = 2
n = Math.trunc(42 / 10) → n = 4

[Iteration 2]

Is n > 0? (4 > 0 is true) → Enter Loop
digit = 4 % 10 → digit = 4
sum = sum + digit → sum = 2 + 4 = 6
n = Math.trunc(4 / 10) → n = 0

[Iteration 3]

Is n > 0? (0 > 0 is false) → EXIT LOOP

Final Output: sum = 6

Part 3 — The Pattern Mastery Bank

Questions are grouped by **algorithmic pattern**, not difficulty — recognizing the pattern is the actual skill.

PATTERN 1

Basic Accumulation & Ranges — "The Bucket" Pattern

Initialize a tracker at 0 or 1, iterate through a clean range, and collect values into it.

1. Print all numbers from 1 to **n** and find their total sum.
2. Calculate the product of all numbers from 1 to **n** (Factorial).
3. Sum only the numbers within a range **[a, b]** divisible by 3 or 5.
4. Count how many numbers between 1 and **n** are multiples of 4 but not multiples of 10.
5. Create an alternating series sum: $1 - 2 + 3 - 4 + 5 \dots \pm n$.

PATTERN 2

Digit Manipulation — "The Scissor & Eraser" Pattern

Use `% 10` and `Math.trunc(n / 10)` to peel numbers apart one digit at a time.

6. Count the total number of digits inside a given integer `n`.
7. Find the sum of all digits of a given number (e.g. 123 → 6).
8. Find the product of all digits of a given number (e.g. 345 → 60).
9. Reverse an integer completely without converting it to a string (e.g. 1234 → 4321).
10. Check if a number is a Palindrome (reads the same forwards and backwards).
11. Check if a number is an Armstrong Number (sum of digits raised to the power of digit count equals the original number).
12. Find the largest single digit inside a multi-digit number.
13. Find the smallest single digit inside a multi-digit number.
14. Count how many times the digit '0' appears inside a number `n`.
15. Extract the first (most significant) digit of any given number. *Hint: loop to shrink until $n < 10$.*

PATTERN 3

Number Theory & Factors — "The Divisor" Pattern

Loop from 1 up to `n` (or $\sqrt{n}$) using `n % i === 0` to scan for clean factors.

16. Find and print all proper divisors of a number `n`.
17. Find the sum of all proper factors of a number and check if it is a Perfect Number.
18. Check if a number is a Prime Number (divisible only by 1 and itself).
19. Print all Prime Numbers that fall between 1 and 100.
20. Find the Highest Common Factor (HCF / GCD) of two numbers.
21. Find the Lowest Common Multiple (LCM) of two numbers using the relationship between HCF and LCM.
22. Check if a number is a Strong Number (sum of the factorial of each digit equals the original number).
23. Find the prime factorization of a number (e.g. 12 → 2, 2, 3).

PATTERN 4

State Tracking & Sequence Generation — "The Memory" Pattern

Store historical values in variables outside the loop to calculate the next sequence step.

24. Generate and print the first `n` terms of the Fibonacci sequence.
25. Find the sum of the first `n` terms of the Fibonacci sequence.
26. Generate a Geometric Progression (GP) series given a starting term, common ratio, and length.
27. Check if a given number belongs anywhere inside the Fibonacci sequence.

28. Find the n-th term of a sequence where each term is the sum of the previous three terms (Tribonacci).

PATTERN 5

Nested Logic & Grids — "The Row-Column Matrix" Pattern

An outer loop tracks rows; an inner loop tracks columns or spaces.

29. Print a square grid of stars of size `n`.
30. Print a left-aligned increasing triangle of stars.
31. Print a right-aligned increasing triangle of stars (requires an inner loop for spaces, then an inner loop for stars).
32. Print an inverted triangle of stars (shrinking from `n` down to 1).
33. Print a perfect centered Pyramid of stars.
34. Print a Diamond star pattern by combining a pyramid and an inverted pyramid.
35. Create a multiplication table grid from 1×1 up to 10×10.
36. Print a sequential number triangle:

```
1
2 3
4 5 6
```

37. Check if a 2D array (matrix) is an Identity Matrix.

PATTERN 6

Early Termination & Skipping — "The Control Flow" Pattern

Use `break` to halt a search when a condition is met; use `continue` to skip processing.

38. Scan a sequence of numbers, but immediately halt everything (`break`) if a negative number appears.
39. Print all numbers from 1 to 50, but skip (`continue`) any multiples of 5.
40. Find the very first prime number inside a specific range `[a, b]` and stop searching immediately.
41. Keep running a sum-tracking loop, but `break` the moment the total sum exceeds 100.

PATTERN 7

Array Traversal & Pointers — "The Linear Scan" Pattern

Loop through index positions `0` to `array.length - 1` to process structural data.

42. Find the largest and smallest element inside an unsorted array.
43. Calculate the average of all numbers stored inside an array.
44. Reverse the elements of an array in-place using a loop (swapping the start and end values).
45. Count the frequency of each unique element within an array.
46. Remove duplicate values from a sorted array without using built-in methods like `Set`.

PATTERN 8

String Parsing & Structural Analysis — "The Text Engine" Pattern

Treat a string like an array of characters, inspecting specific index positions sequentially.

47. Count the total number of vowels (a, e, i, o, u) inside a string.
48. Check if a string is a palindrome (ignoring casing and spaces).
49. Reverse a string completely using a decrementing loop.
50. Convert a binary string (e.g. "1011") into its decimal integer equivalent (11) using positional powers of 2.

Part 4 — Your Playbook to Build Permanent Muscle Memory

To work independently without needing to re-learn this in a year, use this exact workflow for **every single question** above:

1. **Paper first:** never open your code editor first. Write down the problem category name (e.g. "Digit Manipulation").
2. **Identify your mechanics:** write down which math helpers you need (`% 10` , `Math.trunc` , `n % i === 0`).
3. **Draft the loop state:** draw a 3-row tracing table like the one shown in Part 2. Run a small test number manually through your mental algorithm.
4. **Code and break:** type it out into JavaScript. Once it passes, intentionally alter the problem parameters to see if your logic holds up under pressure.